

Cahier de charge:

[PAGE DE GARDE]

Titre : cahier de charge d'un Simulateur de Système de Livraison

Professeur : Youssef ES-SAADY

Auteur : Imane ELHANAFI / Simane ETTHABITY

Date : 20/04/2026

Établissement: *Faculte Polydisciplinaire de Taroudant*

1. PRÉSENTATION DU PROJET

• 1.1 Contexte:

Dans le cadre de notre formation en génie informatique ce projet est réalisé pour le module “programmation objet avec C++”, l'objectif de ce projet est de modéliser un système pour une entreprise de livraison ,gérant différent type de colis doivent être acheminé par différent moyenne de transport (vélo,camion,drone,avion) avec des calculs des coûts ,délais et suivie logistique .

Ce projet permet de mettre en pratique des concepts fondamentaux de la POO:

- l'encapsulation.
- l'héritage.
- le polymorphisme.
- surcharge des opérateurs.
- fonctions amies.

Le défi principal est de mettre en œuvre un système d'attribution automatique des transporteurs basé uniquement sur des méthodes polymorphismes ,sans logique conditionnelle explicite sur les types .

• 1.2 Objectifs généraux:

Objectif	Description
Objectif 1	Permettre la création de colis(poids,fragilité,etats)
Objectif 2	Calculer automatiquement le coût et le délai de livraison selon le transporteur choisi
Objectif 3	Assurer le suivi logistique des colis (en attente → en transit → livré)
Objectif 4	Générer un historique et un rapport récapitulatif des livraisons

Objectif 5	Implémenter un algorithme d'attribution automatique SANS condition sur les types (défi polymorphique)
Objectif 6	Produire un code propre, commenté, versionné sur GitHub, accompagné d'un README, d'un rapport PDF et d'une vidéo de démonstration

• 1.3 Périmètre du projet (DANS / HORS scope)

Ce qui est dans le périmètre :

- création de colis .
- Gestion des transporteur.
- Calcule automatique de coût (selon le poids,fragilité,transporteur)
- Calcule automatique de délai selon le transporteur.
- Suivie des états de livraison .
- Attribut automatique du meilleur transporteur.
- Historique simple.
- Rapport texte (coût total, délai de moyenne,nombre de colis par transporteur)
- fichier [README.md](#) (presentation du projet ,instruction d'exécution)
- Versionnage sur GITHUB.

Ce qui est hors le périmètre:

- Interface graphique.
- Application web ou mobile.
- Base de données

• 1.4 Cible :

Le système cible trois types d'utilisateurs simulés dans un contexte console. Le client expéditeur (boutique ou particulier) qui initie les demandes de livraison. L'agence de livraison qui traite les colis, attribue les transporteurs et génère les rapports. Le destinataire qui confirme la réception du colis. Ces trois rôles sont représentés dans le code par des classes distinctes et interagissent avec le système via des menus console.

2. ANALYSE DES BESOINS (niveau métier)

• 2.1 Acteurs du système

♦ Client (Expéditeur) — Acteur principal

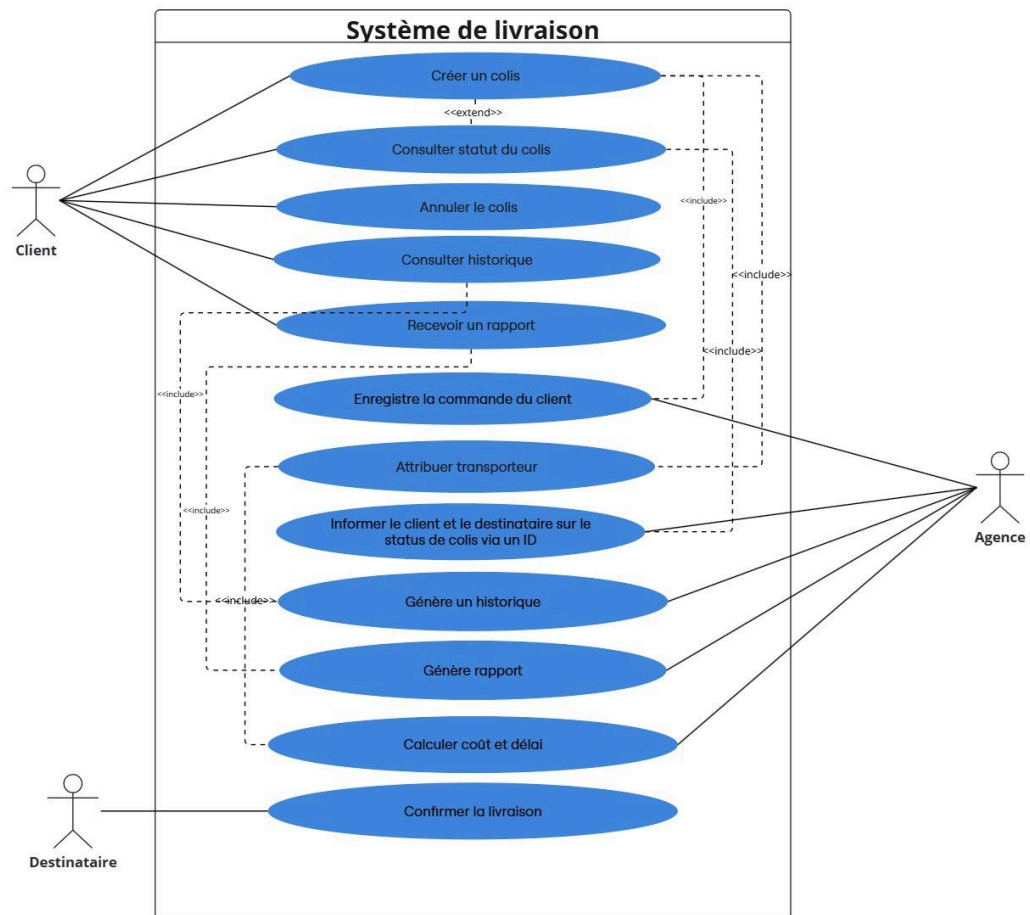
Le Client, aussi appelé Expéditeur, est l'acteur qui initie le processus de livraison. Dans ce contexte, il s'agit typiquement d'une boutique / shop qui reçoit des commandes de ses propres clients (les destinataires). Lorsqu'une commande est passée par un destinataire sur sa boutique, le Client crée une demande de livraison auprès de l'agence en fournissant toutes les informations nécessaires : la distance(entre centre d'agence et l'adresse de destinataire), le type de colis, le poids, les dimensions et le niveau d'urgence.

♦ Agence de livraison — Acteur central (Système)

L'Agence de livraison est l'acteur central et organisateur du système. Elle joue le rôle d'intermédiaire entre le Client (la boutique) et le Destinataire (le receveur final). Dès réception d'une commande du client, l'agence enregistre le colis, lui attribue un identifiant de suivi unique, planifie la collecte auprès du client et organise l'acheminement vers le destinataire. Elle gère l'ensemble du cycle de vie du colis :en attente, en transit , livre .

♦ Destinataire — Acteur final

Le Destinataire est la personne qui a passé une commande auprès de la boutique (le client) et qui attend de recevoir son colis. Il n'a pas de relation contractuelle directe avec l'agence — c'est le client (la boutique) qui a souscrit au service de livraison. Cependant, le destinataire interagit indirectement avec le système : il valide la livraison .



• 2.2 Cas d'utilisation principaux

Client :

- ♦ Créer un colis : fournir les données (type , dimension ...)
- ♦ Consulter le statut de le colis
- ♦ Consulter l'historique
- ♦ Recevoir le rapport
- ♦ Annuler le colis

Agence:

- ♦ Enregistre la commande du client
- ♦ Attribuer tranporteur
- ♦ Calculer coût et délai
- ♦ Informer le client sur le statut de colis via un ID

- ♦ Génère un historique
- ♦ Génère rapport

Destinataire:

- ♦ Confirme la reception de colis

● 2.3 Besoins fonctionnels

Le système de gestion des colis doit couvrir l'ensemble du cycle de vie d'un envoi, de sa création jusqu'à sa livraison.

- ♦ **Côté client**, le système doit permettre de créer un colis en saisissant ses caractéristiques (poids, dimensions, distance, type et urgence), de suivre son état via un identifiant unique, et d'annuler une commande sous certaines conditions. Le client reçoit des messages de confirmation à chaque étape clé.
- ♦ **Côté transport**, le système détermine automatiquement le mode de transport adapté selon les données du colis, et calcule le coût de livraison en tenant compte du poids, de la distance, du type et de l'urgence.
- ♦ **Côté destinataire**, la réception du colis est confirmée via le système, ce qui déclenche l'archivage dans l'historique.
- ♦ **Côté agence**, le système offre la possibilité de consulter les paramètres des colis et de générer des rapports sur les livraisons effectuées.

● 2.4 Règles métier identifiées

1. Création du colis

- ♦ Un colis ne peut être créé que si toutes les informations obligatoires sont renseignées :
 - poids
 - dimensions
 - distance
 - type
 - urgence
- ♦ Chaque colis doit avoir un identifiant unique.
- ♦ Les valeurs saisies doivent être valides :
 - poids > 0

- dimensions > 0
- distance > 0
- type soit : “fragile” ou “standard”
- urgence soit : “true” ou “false”

2. Type de transport

- ♦ Le type de transport est déterminé automatiquement en fonction :

- du poids
- de la distance
- du type (fragile / standard)
- de l'urgence

Transporteur	Conditions d'éligibilité
Vélo	distance $\leq$ 10 km ET poids $\leq$ 5 kg ET non fragile
Drone	poids $\leq$ 3 kg ET distance $\leq$ 30 km ET non fragile
Camion	toujours disponible (cas par défaut)
Avion	distance > 500 km

Transporteur	Délai normal	Délai urgent
Vélo	3 Jours	1 Jour
Camion	5 Jours	1 Jour
Drone	1 Jour	Même jour
Avion	7 Jours	1 Jour

3. Calcul des coûts

- ♦ Le coût de livraison dépend obligatoirement de :
 - poids
 - distance
 - type de colis
 - urgence
- ♦ Le coût doit être calculé avant validation de la commande
- ♦ Chaque transporteur a son propre coefficient de coût de base

Transporteur	Formule de base	Supplément urgence (multiplicateur)
Vélo	$\text{poids} \times 2 + \text{distance} \times 0.5$	$\times 1.5$
Camion	$\text{poids} \times 1.5 + \text{distance} \times 0.3$	$\times 1.3$
Drone	$\text{poids} \times 4 + \text{distance} \times 1.2$	$\times 2.0$
Avion	$\text{poids} \times 3 + \text{distance} \times 0.8$	$\times 1.8$

4. Suivi du colis

- ♦ Le suivi d'un colis nécessite un ID valide
- ♦ Un utilisateur ne peut pas consulter que :
 - les informations liées à son colis
- ♦ Chaque colis possède un statut (ex : en attente, en cours, livré, annulé)
- ♦ Un colis à l'état livré ne peut plus être modifié ni annulé

5. Confirmation de livraison

- ♦ Seul le destinataire peut confirmer la livraison
- ♦ Une fois confirmée :
 - le statut devient "livré"
 - le colis est ajouté à l'historique
- ♦ Une livraison confirmée est irréversible.

6. Annulation du colis

- ♦ Le client peut annuler le colis si :
 - il n'a pas encore été traité par l'agence → statut = en_attente

7. Historique

- ♦ Toute action sur un colis doit être enregistrée :
 - création
 - modification
 - annulation
 - status de livraison
- ♦ L'historique ne doit jamais être supprimé.
- ♦ L'historique est consultable par ID de colis

8. Rapports

- ♦ Le rapport final doit contenir : ID_colis , coût_total , type_transporteur , poids , dimension , type , urgence et distance .
- ♦ Seule l'agence peut :
 - générer
- ♦ Le client reçoit le rapport

9. Gestion des rôles

- ♦ Le système distingue trois rôles :
 - client
 - agence
 - destinataire
- ♦ Chaque rôle a des droits spécifiques :
 - client → créer / suivre / annuler / reçoit le rapport
 - agence → valider / calculer / génère le rapport
 - destinataire → confirmer la livraison

- ## 2.5 Hypothèses

1. Disponibilité du système

- ♦ Le système fonctionne en local sur un PC disposant d'un compilateur C++ et de CMake.
- ♦ Les utilisateurs disposent d'un appareil compatible (PC).

2. Rôles des utilisateurs

- ♦ Chaque utilisateur connaît son rôle :
 - client
 - agence
 - destinataire
- ♦ Aucun utilisateur ne tente d'accéder à des fonctionnalités non autorisées.

3. Processus de livraison

- ♦ Une fois le type de transport choisi, le processus de livraison se déroule sans interruption externe.
- ♦ L'agence traite les colis dans un délai raisonnable.

4. Annulation

- ♦ Le client ne cherche pas à exploiter le système avec des annulations abusives.
- ♦ Les conditions d'annulation sont respectées par tous les utilisateurs.

5. Identification des colis

- ♦ Chaque colis est identifié par un ID unique fiable.
- ♦ Les utilisateurs utilisent toujours cet ID correctement pour le suivi.

6. Portée du système

- ♦ Le système ne gère pas :
 - les paiements en ligne

-les problèmes physiques de livraison (perte, casse réelle)

- ♦ Il se concentre uniquement sur la gestion et le suivi informatique

3. EXIGENCES FONCTIONNELLES (*niveau système*)

- 3.1 Liste des fonctions (EF-01, EF-02...)

ID	Description	Priorité
EF-01	Le système doit permettre au client de créer un colis (poids, dimensions, distance, type, urgence)	Obligatoire
EF-02	Le système doit générer automatiquement un identifiant unique (ID) pour chaque colis	Obligatoire
EF-03	Le système doit valider les données saisies (poids > 0, dimensions > 0, type valide...)	Obligatoire
EF-04	Le système doit afficher un message de confirmation après la création du colis	Obligatoire
EF-05	Le système doit déterminer automatiquement le type de transporteur selon les règles définies en 2.4 point 2 .	Obligatoire
EF-06	Le système doit calculer le coût de livraison (poids, distance, type, urgence, transporteur)	Obligatoire
EF-07	Le système doit calculer le délai de livraison selon le transporteur	Obligatoire
EF-08	Le système doit permettre la consultation du statut d'un colis via son ID	Obligatoire
EF-09	Le système doit gérer les transitions d'état : en attente → en transit → livré → annulé	Obligatoire
EF-10	Le système doit permettre au destinataire de confirmer la réception du colis	Obligatoire
EF-11	Le système doit ajouter automatiquement un colis à l'historique après confirmation de livraison	Obligatoire
EF-12	Le système doit permettre au client d'annuler un colis non encore traité par l'agence	Obligatoire
EF-13	Le système doit enregistrer toutes les actions (création, modification, annulation, livraison)	Obligatoire
EF-14	Le système doit permettre à l'agence de générer un rapport (ID, coût total, délai moyen, transporteur, type...)	Obligatoire
EF-15	Le système doit afficher des messages d'erreur clairs pour toute saisie	Optionnel

	invalide ou action interdite .	
EF-16	Le système doit permettre la consultation de l'historique par ID de colis .	Optionnel

• 3.2 Priorités

Les fonctions EF-01 à EF-14 sont obligatoires (Must Have).

Les fonctions EF-15 et EF-16 sont optionnelles (Could Have).

Aucune fonctionnalité n'est classée "Won't Have" dans cette version.

4. EXIGENCES NON FONCTIONNELLES

• 4.1 Contraintes techniques

ID	Catégorie	Description
CT-01	Langage	Le système doit être développé exclusivement en C++
CT-02	POO	Le code doit obligatoirement utiliser les classes, l'héritage et l'encapsulation ainsi que le polymorphisme
CT-03	Polymorphisme	Une classe abstraite <code>Transporteur</code> doit être définie avec des méthodes virtuelles pures <code>computeCost()</code> , <code>computeDelay()</code> , <code>canDeliver()</code>
CT-04	Polymorphisme	L'attribution du transporteur doit se faire sans condition explicite sur les types (if/switch)
CT-05	Encapsulation	Tous les attributs des classes doivent être privés, accessibles uniquement via getters/setters
CT-06	Héritage	Une classe abstraite <code>Personne</code> doit être héritée par <code>Client_agence</code> et <code>Destinataire</code>
CT-07	Gestion des erreurs	Le système doit gérer toutes les exceptions (saisies invalides, ID incorrect, annulation interdite) sans arrêt du programme
CT-08	Identifiants	Chaque colis doit avoir un ID unique généré automatiquement, sans doublon
CT-09	Validation	Toutes les entrées utilisateur doivent être validées avant traitement (poids > 0, distance > 0, type valide...)
CT-10	Performance	Le système doit répondre à toute requête (création, suivi, calcul) en moins de 2 secondes
CT-11	Performance	L'affichage de l'historique et des rapports doit se faire en moins d'une seconde
CT-12	Sécurité	L'accès aux fonctionnalités doit être restreint selon le rôle (client / agence / destinataire)
CT-13	Fiabilité	L'algorithme d'attribution automatique doit toujours retourner un transporteur valide
CT-14	Scalabilité	L'architecture du code doit permettre l'ajout futur d'une base de

		données ou d'une interface graphique sans refactoring majeur
CT-15	Utilisabilité	Interface simple et claire pour un client non technique
CT-16	Traçabilité	Historique consultable par ID

• 4.2 Contraintes de livrables

ID	Livrable	Description
CL-01	Code source	Le code C++ complet, structuré et modulaire (un fichier par classe)
CL-02	Code source	Le code doit être commenté : explication des méthodes, du polymorphisme et des choix de conception
CL-03	GitHub	Le projet doit être versionné sur un dépôt GitHub public avec historique de commits
CL-04	README.md	Un fichier README.md doit contenir : présentation du projet, structure des fichiers, instructions de compilation et d'exécution
CL-05	Rapport PDF	Un rapport PDF complet incluant : contexte, analyse des besoins, conception (diagrammes UML), implémentation et conclusion
CL-06	Diagrammes UML	Le rapport doit inclure au minimum : diagramme de classes, diagramme de cas d'utilisation
CL-07	Vidéo	Une vidéo de démonstration du fonctionnement de l'application (exécution console)
CL-08	Rapport –Statistique	Le rapport doit contenir : coût total des livraisons, délai moyen, nombre de colis par transporteur

5. CONTRAINTES DU PROJET

• 5.1 Délais

ID	Description	Échéance	Priorité
CDP-01	Rendu du rapport PDF complet	Fin du semestre	Obligatoire

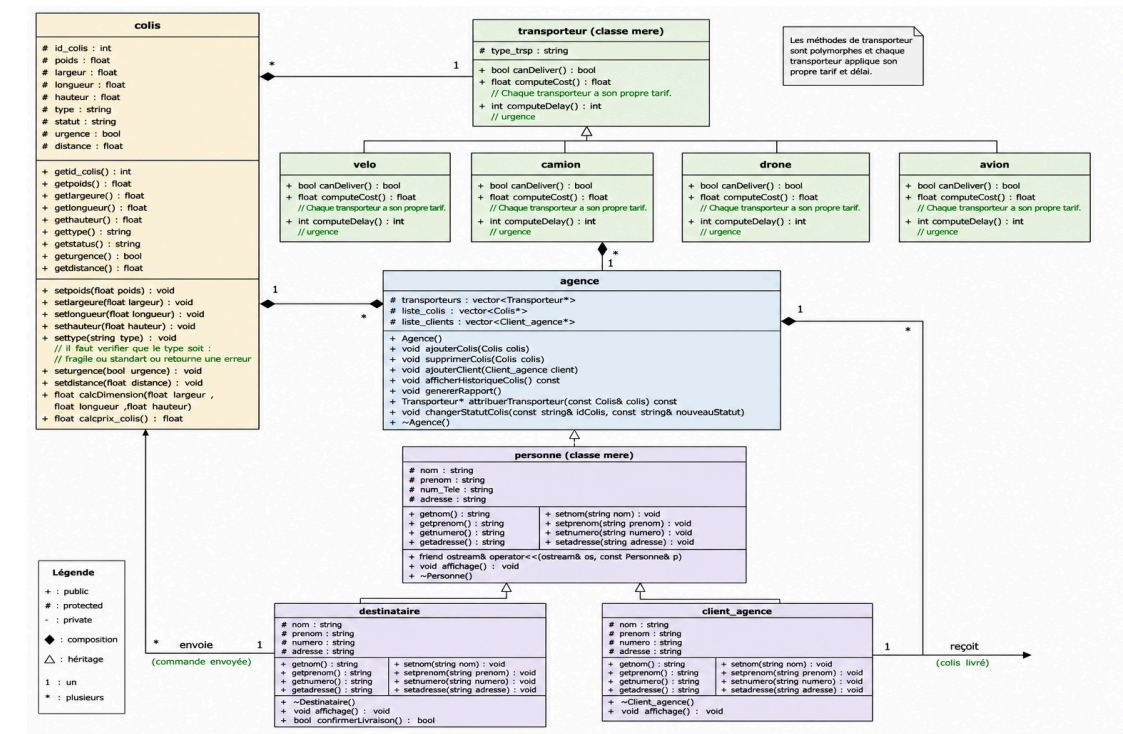
CDP-02	Rendu du code source final sur GitHub	Fin du semestre	Obligatoire
CDP-03	Rendu de la vidéo de démonstration	Fin du semestre	Obligatoire

- 5.2 Technologies imposées

ID	Technologie	Détail
CTI-01	Langage	C++ (standard C++11 minimum)
CTI-02	Paradigme	Programmation Orientée Objet (POO) obligatoire
CTI-03	Concepts POO	Encapsulation, héritage, polymorphisme, surcharge d'opérateurs, fonctions amies
CTI-04	Build System	CMake (version 3.10 minimum recommandée)
CTI-05	Interface	Console uniquement (pas d'interface graphique ni web)
CTI-06	Stockage	Pas de base de données — structures en mémoire uniquement (tableaux, vecteurs, listes)
CTI-07	Versionnage	Git + GitHub obligatoire
CTI-08	Documentation	README.md en Markdown
CTI-9	Rapport	Format PDF uniquement

6. CONCEPTION

6.1 Diagramme de classes



7. GLOSSAIRE

7.1 Termes Métier

Terme	Définition
Colis	Objet physique à livrer, identifié par un ID unique, un poids, des dimensions, un type et un niveau d'urgence
Client	Boutique ou expéditeur qui crée une demande de livraison et paie les frais à l'agence
Destinataire	Personne qui reçoit le colis et confirme sa réception
Agence/Systeme	Entité centrale qui reçoit, enregistre, attribue et suit les colis

Transporteur	Moyen utilisé pour acheminer un colis : vélo, camion, drone ou avion
Livraison	Processus d'acheminement d'un colis de l'agence jusqu'au destinataire
Coût de livraison	Montant calculé automatiquement selon le poids, la distance, le type et l'urgence du colis
Délai de livraison	Durée estimée pour livrer un colis selon le transporteur assigné
Urgence	Indicateur signalant qu'un colis doit être livré en priorité
Fragilité	Caractéristique indiquant qu'un colis nécessite une manipulation délicate
Annulation	Autorisée uniquement si le statut du colis est EN_ATTENTE (non encore traité par l'agence). Bloquée si statut = EN_TRANSIT ou LIVRÉ
Confirmation de livraison	Validation de la réception effectuée par le destinataire — action irréversible
Historique	Enregistrement permanent de toutes les actions effectuées sur les colis
Rapport	Document récapitulatif généré par l'agence (coût total, délai moyen, nombre de colis par transporteur)

● 7.2 status d'un colis

Terme	Définition
En attente	Colis créé mais pas encore pris en charge par l'agence
En transit	Colis en cours d'acheminement vers le destinataire

Livré	Colis confirmé comme reçu par le destinataire — statut irréversible
Annulé	Colis dont la commande a été annulée selon les conditions définies

• 7.3 termes techniques

Terme	Définition
Classe	Structure de base en C++ regroupant des attributs (données) et des méthodes (comportements)
Classe abstraite	Classe qui ne peut pas être instanciée directement — elle impose à ses classes filles de redéfinir certaines méthodes
Objet	Instance d'une classe — par exemple, un colis précis est un objet de la classe Colis
Héritage	Mécanisme permettant à une classe fille de réutiliser les attributs et méthodes d'une classe mère
Polymorphisme	Capacité d'une méthode à se comporter différemment selon le type d'objet — ici chaque transporteur calcule son coût à sa façon
Encapsulation	Principe qui consiste à cacher les données d'une classe (attributs privés) et à n'y accéder que via des méthodes publiques
Méthode virtuelle pure	Méthode déclarée <code>= 0</code> dans une classe abstraite, obligatoirement redéfinie dans chaque sous-classe
Surcharge d'opérateur	Redéfinition d'un opérateur C++ (<code><<</code> , <code>==</code> , <code>+</code> ...) pour l'adapter au comportement d'une classe
Fonction amie	Fonction externe déclarée <code>friend</code> dans une classe, lui donnant accès à ses membres privés

Getter	Méthode publique qui retourne la valeur d'un attribut privé
Setter	Méthode publique qui modifie la valeur d'un attribut privé
ID unique	Identifiant généré automatiquement pour chaque colis, garantissant qu'aucun doublon n'existe
Exception	Erreur détectée à l'exécution (ex : poids négatif) gérée par le programme sans provoquer son arrêt

● 7.4 Outils et Technologies

Terme	Définition
C++	Langage de programmation orienté objet utilisé pour développer ce projet
CMake	Outil de build qui automatise la compilation d'un projet C++ à partir d'un fichier CMakeLists.txt
CMakeLists.txt	Fichier de configuration CMake décrivant les sources, les règles de compilation et la structure du projet
g++	Compilateur GNU qui transforme le code source C++ en programme exécutable
Git	Outil de versionnage qui enregistre l'historique des modifications du code source
GitHub	Plateforme en ligne qui héberge le dépôt Git du projet et facilite la collaboration
README.md	Fichier de présentation du projet, rédigé en Markdown, décrivant le projet et les instructions d'exécution
Markdown	Langage de mise en forme simple utilisé pour rédiger des fichiers texte comme le README

7.5 Acronymes

Acronyme	Signification	Définition courte
POO	Programmation Orientée Objet	Paradigme de développement basé sur des classes et des objets
UML	Unified Modeling Language	Langage standard de modélisation utilisé pour créer des diagrammes de conception
ID	Identifiant	Code unique attribué automatiquement à chaque colis
EF	Exigence Fonctionnelle	Ce que le système doit faire
ENF	Exigence Non Fonctionnelle	Comment le système doit le faire (performance, sécurité...)
CT	Contrainte Technique	Règle technique imposée au développement
CL	Contrainte de Livrable	Document ou fichier à rendre obligatoirement

8. ANNEXES

Simulateur de Système de Livraison/

main

└─ Agence.cpp / Agence.h

└─ Avion.cpp / Avion.h

└─ Camion.cpp / Camion.h

└─ Client_agence.cpp / Client_agence.h

- |— Destinataire.cpp / Destinataire.h
- |— Drone.cpp / Drone.h
- |— Personne.cpp / Personne.h
- |— Transporteur.h
- |— Velo.cpp / Velo.h
- |— colis.cpp / colis.h
- |— interfaceConsole.cpp / interfaceConsole.exe
- |— std.o
- └─ README.md
- |
- |— feat/transport ← fichiers transport
- | |— Avion.cpp / Avion.h
- | |— Camion.cpp / Camion.h
- | |— Drone.cpp / Drone.h
- | |— Transporteur.h
- | |— Velo.cpp / Velo.h
- | |└─ README.md
- |
- |— feat/colis ← fichiers colis
- | |— colis.cpp / colis.h
- | |— colis.exe
- | |— main.cpp
- | |— .gitignore
- | |└─ README.md
- |
- └─ dev ← branche de test (tout)
 - |— (tous les fichiers de main)
 - |— + tests avant push vers main
 - └─ merge depuis feat/*

feat/transport —┐

└─→ dev (tests) —→ main

feat/colis —┐